

-  GUÍA DE INTEGRACIÓN — FRANCISCO
 -  ARCHIVOS QUE SON TU RESPONSABILIDAD
 -  ¿QUÉ HACE CADA SCRIPT?
 -  PRE-REQUISITOS
 -  PASO A PASO COMPLETO
 - PASO 1 — Clonar el repositorio (solo si no lo tienes)
 - PASO 2 — Configurar identidad Git
 - PASO 3 — Estar en develop actualizada
 - PASO 4 — Crear tu rama de trabajo
 - PASO 5 — Verificar tus archivos
 - PASO 6 — Agregar SOLO tus archivos al staging
 - PASO 7 — Ver qué cambios incluyes
 - PASO 8 — Hacer commit
 - PASO 9 — Subir tu rama a GitHub
 - PASO 10 — Verificar en GitHub
 - PASO 11 — Crear el Pull Request
 -  AGREGAR MÁS SCRIPTS DESPUÉS
 -  PROBLEMAS COMUNES Y SOLUCIONES
 - "railway.exe is too large to push"
 - Los scripts .bat no están rastreados por Git
 - El script aparece con encoding incorrecto en GitHub
 - Error "LF will be replaced by CRLF"
 -  CHECKLIST PERSONAL — FRANCISCO
 - Preparación
 - Antes del commit
 - PR
 -  INFORMACIÓN TÉCNICA

GUÍA DE INTEGRACIÓN — FRANCISCO

Componentes a cargo: Scripts de inicio/parada del sistema **Rama de trabajo:** `feature/francisco-scripts` **Ejecutar en:** FASE 1 (¡el primero en subir!)

ARCHIVOS QUE SON TU RESPONSABILIDAD

```
Eventos-Microservicios/  (raíz del proyecto)
├── Start-All.ps1          ← Script principal de inicio
├── Stop-All.ps1           ← Script principal de parada
├── Start-Local.ps1        ← Script de inicio local (dev)
├── Monitor-Services.ps1   ← Dashboard de monitoreo
├── Iniciar-Proyecto.bat    ← Lanzador Windows (.bat) para inicio
└── Detener-Proyecto.bat    ← Lanzador Windows (.bat) para parada
```

 Estos archivos están en la **raíz del proyecto** (no dentro de subcarpetas). Tu tarea es la más independiente: no depende de ningún otro servicio.

¿QUÉ HACE CADA SCRIPT?

Script	Función
<code>Start-All.ps1</code>	Inicia MySQL Docker, compila servicios, lanza todos los 7 componentes
<code>Stop-All.ps1</code>	Detiene todos los procesos Java y Node.js en puertos 8080-8085 + 5173
<code>Start-Local.ps1</code>	Versión simplificada para desarrollo local
<code>Monitor-Services.ps1</code>	Panel visual que muestra el estado de cada servicio en tiempo real
<code>Iniciar-Proyecto.bat</code>	Doble-clic en Windows → ejecuta <code>Start-All.ps1</code> con PowerShell
<code>Detener-Proyecto.bat</code>	Doble-clic en Windows → ejecuta <code>Stop-All.ps1</code> con PowerShell

PRE-REQUISITOS

```
# Verificar Git instalado
git --version
# Resultado esperado: git version 2.x.x

# Verificar PowerShell (para probar los scripts)
powershell --version
# Resultado esperado: PowerShell 5.x o superior (Windows nativo)

# Verificar que puedes ejecutar scripts PowerShell
powershell -Command "Get-ExecutionPolicy"
# Si dice "Restricted", los .bat ya lo manejan con -ExecutionPolicy Bypass
```



PASO A PASO COMPLETO

PASO 1 — Clonar el repositorio (solo si no lo tienes)

```
git clone [URL_DEL_REPO]
cd Eventos-Microservicios
```

PASO 2 — Configurar identidad Git

```
git config user.name "Francisco"
git config user.email "tu-email@gmail.com"
```

PASO 3 — Estar en develop actualizada

✅ Tú eres el PRIMERO en subir, así que **develop** puede estar limpia o con pocos archivos.

```
git checkout develop
git pull origin develop
```

PASO 4 — Crear tu rama de trabajo

```
git checkout -b feature/francisco-scripts
```

Confirma:

```
git branch  
# * feature/francisco-scripts
```

PASO 5 — Verificar tus archivos

Comprueba que los scripts existen:

```
ls -la *.ps1 *.bat
```

Debes ver:

```
Start-All.ps1  
Stop-All.ps1  
Start-Local.ps1  
Monitor-Services.ps1  
Iniciar-Proyecto.bat  
Detener-Proyecto.bat
```

PASO 6 — Agregar SOLO tus archivos al staging

 **IMPORTANTE:** Agrega los archivos uno por uno o con patrones específicos.

Nunca `git add .`

```
# Agregar scripts PowerShell  
git add Start-All.ps1  
git add Stop-All.ps1  
git add Start-Local.ps1  
git add Monitor-Services.ps1
```

```
# Agregar scripts batch
git add Iniciar-Proyecto.bat
git add Detener-Proyecto.bat

# VERIFICAR inmediatamente
git status
```

Salida esperada:

```
On branch feature/francisco-scripts
Changes to be committed:
  modified:   Start-All.ps1
  modified:   Stop-All.ps1
  modified:   Start-Local.ps1
  modified:   Monitor-Services.ps1
  modified:   Iniciar-Proyecto.bat
  modified:   Detener-Proyecto.bat
```

```
# NO debe aparecer nada de: auth-service/, event-service/, etc.
```

✦ Si aparece **railway.exe** o **railway.zip** y forman parte de tus entregables, también puedes agregarlos:

```
git add railway.exe
git add railway.zip
```

⚠ Ten cuidado con archivos binarios grandes. Si **railway.zip** tiene más de 50MB, GitHub lo rechazará. En ese caso, NO lo agregues y avisa a Rachel.

PASO 7 — Ver qué cambios incluyes

```
# Para archivos de texto (scripts), puedes ver los cambios
git diff --staged Start-All.ps1

# Para archivos binarios (.bat son texto plano en realidad)
git diff --staged Iniciar-Proyecto.bat
```

PASO 8 — Hacer commit

```
git commit -m "feat(scripts): scripts de inicio, parada y monitoreo del sistema"
```

Start-All.ps1:

- Verifica prerequisites (Java 17+, Node.js)
- Compila microservicios con Maven Wrapper
- Lanza 6 microservicios + frontend en background
- Conecta a Railway MySQL (shinkansen.proxy.rlwy.net:37791)
- Guarda logs en .logs/

Stop-All.ps1:

- Detiene todos los procesos Java en puertos 8080-8085
- Detiene Node.js en puerto 5173

Monitor-Services.ps1:

- Dashboard visual de estado de servicios
- Actualización cada 5 segundos
- Espera hasta 120s para que todos los servicios inicien

Iniciar-Proyecto.bat / Detener-Proyecto.bat:

- Launchers de doble-clic para Windows"

PASO 9 — Subir tu rama a GitHub

Primera vez:

```
git push -u origin feature/francisco-scripts
```

Si hay cambios adicionales:

```
git push origin feature/francisco-scripts
```

Force push si es necesario:

```
git push origin feature/francisco-scripts --force-with-lease
```

PASO 10 — Verificar en GitHub

1. Abre GitHub → repositorio
2. Selecciona rama `feature/francisco-scripts`

3. Verifica que solo aparecen: `Start-All.ps1`, `Stop-All.ps1`, `Start-Local.ps1`, `Monitor-Services.ps1`, `Iniciar-Proyecto.bat`, `Detener-Proyecto.bat`
4. Abre uno de los archivos y confirma que el contenido es correcto

PASO 11 — Crear el Pull Request

Por GitHub UI:

1. Clic en **"Compare & pull request"**
2. Configura:
 - **Base:** `develop`
 - **Compare:** `feature/francisco-scripts`
 - **Título:** `feat(scripts): scripts de inicio, parada y monitoreo del sistema`
 - **Descripción:**

```
## Scripts incluidos

### PowerShell
- `Start-All.ps1` - Inicio completo del sistema (compila + lanza todos los servicios)
- `Stop-All.ps1` - Parada completa del sistema
- `Start-Local.ps1` - Inicio para desarrollo local
- `Monitor-Services.ps1` - Dashboard de monitoreo en tiempo real

### Batch (launchers Windows)
- `Iniciar-Proyecto.bat` - Doble clic para iniciar
- `Detener-Proyecto.bat` - Doble clic para detener

## Configuración de base de datos
Railway MySQL: shinkansen.proxy.rlwy.net:37791

## Pruebas realizadas
- [ ] Start-All.ps1 ejecuta sin errores de sintaxis
- [ ] Stop-All.ps1 ejecuta sin errores de sintaxis
- [ ] Los .bat abren PowerShell correctamente
```

3. Clic **"Create Pull Request"**
4. **Notifica a Rachel** que tu PR está listo — ella será la primera en revisar

Por CLI:

```
gh pr create \
  --base develop \
```

```
--head feature/francisco-scripts \  
--title "feat(scripts): scripts de inicio, parada y monitoreo del sistema" \  
--body "Scripts PowerShell y BAT para gestión completa del ciclo de vida del  
sistema de microservicios"
```



AGREGAR MÁS SCRIPTS DESPUÉS

Si necesitas agregar un nuevo script después:

```
# Asegúrate de estar en tu rama  
git checkout feature/francisco-scripts  
  
# Agrega el nuevo script  
git add NuevoScript.ps1  
  
# Commit  
git commit -m "feat(scripts): agrego NuevoScript.ps1 para [propósito]"  
  
# Push (el PR se actualiza automáticamente)  
git push origin feature/francisco-scripts
```



PROBLEMAS COMUNES Y SOLUCIONES

"railway.exe is too large to push"

GitHub tiene un límite de 100MB por archivo. Si **railway.exe** o **railway.zip** son muy grandes:

```
# Verificar tamaño  
ls -lh railway.exe railway.zip  
  
# Si son > 50MB, NO los subas a GitHub  
# En vez de eso, añádelos al .gitignore  
echo "railway.exe" >> .gitignore  
echo "railway.zip" >> .gitignore  
  
# O usa Git LFS (habla con Rachel)
```



```
git lfs track "*.exe"
git lfs track "*.zip"
```

Los scripts .bat no están rastreados por Git

```
# Verificar
git status
# Si aparecen como "untracked files", es normal, solo agrégalos:
git add Iniciar-Proyecto.bat
git add Detener-Proyecto.bat
```

El script aparece con encoding incorrecto en GitHub

Los scripts PowerShell deben estar en UTF-8. Si ves caracteres extraños en GitHub:

```
# Verificar encoding
file Start-All.ps1
# Resultado esperado: UTF-8 text

# Si está en UTF-16 u otro, conviértelo con tu editor (VS Code → Save with Encoding
→ UTF-8)
```

Error "LF will be replaced by CRLF"

```
# Este es un warning, no un error. Es normal en Windows
# Si quieres evitarlo:
git config core.autocrlf true
# Esto convierte automáticamente los line endings
```



CHECKLIST PERSONAL — FRANCISCO

Preparación

- ☐ Git configurado con mi nombre y email
- ☐ `git checkout develop && git pull origin develop`
- ☐ Rama `feature/francisco-scripts` creada
- ☐ Los 6 archivos existen en la raíz del proyecto

Antes del commit

- ☐ Solo los scripts específicos están en staging
- ☐ `railway.exe` / `railway.zip` manejados correctamente (size check)
- ☐ Mensaje de commit descriptivo

PR

- ☐ Push exitoso a `feature/francisco-scripts`
- ☐ Verificado en GitHub que los archivos son correctos
- ☐ **Base:** `develop` ✓
- ☐ PR creado y notificado a Rachel
- ☐ Rachel sabe que soy el primero — espero su confirmación de merge antes de hacer nada más



INFORMACIÓN TÉCNICA

Configuración de Railway MySQL en los scripts:

```
Host:      shinkansen.proxy.rlwy.net
Port:      37791
User:      root
Password:  UtYYbWKKZnfIGkALRiPnDnsLuXPLYQzj
DBs:      auth_db, user_db, event_db, inscrip_db, notification_db
```

Puertos que gestiona el sistema:

```
8080 - API Gateway
8081 - auth-service
8082 - user-service
8083 - event-service
8084 - inscrip-service
8085 - notification-service
5173 - frontend-v2
```

Start-All.ps1 inicia los servicios en este orden:

1. Verifica Java 17+ y Node.js
2. Compila JARs con Maven (si no existen)
3. Lanza auth-service (8081)
4. Lanza user-service (8082)
5. Lanza event-service (8083)
6. Lanza inscrip-service (8084)
7. Lanza notification-service (8085)
8. Lanza gateway (8080)
9. Lanza frontend-v2 (5173)
10. Ejecuta Monitor-Services.ps1